

Fit33 Codebase Audit Report

Date: 2026-02-25

Scope: Read-only static review of iOS app (`Fit33`) + Supabase SQL/Edge Functions.

Constraint followed: no source code changes made.

1) Executive Summary

Fit33 is a feature-rich SwiftUI fitness app with strong product depth (workouts, social, challenges, nutrition, and integrations), but it is currently carrying **high operational and maintenance risk** due to:

- hardcoded secrets in app code and edge-function comments,
- very large multi-responsibility files (multiple 5k-8k+ line files),
- partially implemented user-critical flows (shared workout deep links),
- backend schema drift risk from many overlapping SQL “fix” scripts,
- low automated test coverage relative to complexity.

This is not a quality problem in one module; it is mostly a **systems hygiene and scale-management problem**.

2) App Architecture and How Elements Work Together

Frontend / App Layers

- **Entry & lifecycle:** `Fit33App.swift` orchestrates startup, auth checks, notifications, realtime, performance staging, deep links, and scene-phase background/foreground behavior.
- **Root routing:** `ContentView.swift` decides between onboarding (`NewOnboardingView`) and app tabs (`MainTabView`) based on `UserManager.hasCompletedOnboarding`.
- **Primary domains:**
 - **Workouts:** `WorkoutManager`, `WorkoutTabView`, `ActiveWorkoutView`, generator engines (`WorkoutGeneratorService`, `SmartProgramEngine`, `SmartExerciseSelectionEngine`).
 - **Social/Challenges:** `FriendService`, `ChallengeService`, `RealtimeService`, `FriendsTabView`, challenge views.
 - **Nutrition:** `MealService`, `FoodDatabaseService`, `FoodSearchView`, recipe services.
 - **Health integrations:** `HealthKitService`, `HealthKitManager`, `HealthDataService`, Strava/Fitbit/InBody services.
 - **Persistence mix:** Core Data for local state + Supabase for cloud/auth/realtime + UserDefaults for state flags/caches.

Backend / Supabase

- **SQL strategy:** many standalone SQL scripts in `supabase/`, including several large one-off “fix” scripts and repeated function redefinitions.
- **Edge functions:** push notifications, contact-joined notifications, Twilio verification, USDA food proxy.

Observed coupling pattern

- Significant use of singleton services (`shared`) and cross-service side effects.
- Many domain flows are coordinated through app lifecycle hooks and NotificationCenter.

3) Priority Findings (What is wrong, why, and recommended fix)

Critical

C1. Hardcoded secrets and credentials in repo

What is wrong

- `Fit33/AppConfig.swift` contains live API secrets (Spoonacular key, Strava client secret, Fitbit client secret).
- `Fit33/SupabaseManager.swift` contains Supabase URL + anon key in code.
- `supabase/functions/usda-food-search/index.ts` comment includes a concrete USDA API key example.

Why it is wrong

- Keys in client code and repo are easy to extract and abuse (quota exhaustion, account compromise, data exposure, app impersonation).

Recommended fix

1. Rotate exposed keys immediately.
2. Move all third-party secrets server-side where possible (Edge Functions/secure backend).
3. Keep only public identifiers in client app; fetch ephemeral/limited tokens if needed.
4. Add secret scanning pre-commit/CI (e.g., gitleaks/trufflehog).

C2. Shared workout deep-link flow is functionally incomplete

What is wrong

- `WorkoutSharingService.loadSharedWorkout` and `SharedWorkoutView.loadWorkout` still rely on local storage and contain TODO for server fetch.
- Shared links can fail for workouts created on other accounts/devices.

Why it is wrong

- The user-visible promise “share workout link” is not reliably delivered cross-user, causing broken core social behavior.

****Recommended fix****

1. Create a canonical cloud `shared\_workouts` read path and access policy.
2. Resolve deep links by cloud lookup first, local fallback second.
3. Add analytics + graceful UX state (“link expired/private/not found”).
4. Add one end-to-end test case for cross-account shared workout open.

C3. Backend migration/process drift risk from overlapping SQL scripts

****What is wrong****

- 31 SQL scripts with many repeated function definitions (14 duplicated function names across files).
- Multiple scripts drop/recreate same functions (`get\_active\_group\_challenges`, `cancel\_group\_challenge`, `log\_challenge\_progress`, etc.).
- Deployment process is manual order-driven via docs (`DEPLOYMENT\_ORDER.md`), not a single authoritative migration chain.

****Why it is wrong****

- Environment drift and order-sensitive regressions become likely.
- Hotfix scripts can overwrite prior behavior unintentionally.

****Recommended fix****

1. Move to timestamped immutable migrations (single source of truth).
2. Consolidate repeated function definitions into canonical latest versions.
3. Add a schema diff check between local/prod and run it in CI.
4. Require transactional migration blocks for non-trivial schema changes.

High

H1. Extremely large files and mixed responsibilities

****What is wrong****

- Largest Swift files include: `DashboardView.swift` (8727 lines), `NewOnboardingView.swift` (8622), `ContentView.swift` (5299), `SupabaseManager.swift` (5155), etc.

****Why it is wrong****

- High regression probability, slow onboarding for developers, hard testability, fragile merge behavior.

****Recommended fix****

- Extract per-domain coordinators/components (auth flow, deep-link flow, startup pipeline, profile sync, etc.) and keep file/module boundaries strict.

H2. Startup/auth duplication and possible redundant sync work

****What is wrong****

- `SupabaseManager`` calls `checkAuth()` in its init and app root also calls `await supabaseManager.checkAuth()` in startup task.

****Why it is wrong****

- Duplicate auth/session + sync paths can increase startup load and produce racey state changes.

****Recommended fix****

- Define one startup auth coordinator with idempotent check/sync execution and explicit phase ownership.

H3. Very high logging volume in production paths

****What is wrong****

- ~3,485 `print(...)` calls across Swift sources; many include detailed flow/user metadata.

****Why it is wrong****

- Performance overhead, noise, potential privacy leakage in logs, hard incident triage.

****Recommended fix****

- Route logs through a level-based logger; strip or redact PII; gate verbose logs to debug builds.

H4. Navigation architecture inconsistency

****What is wrong****

- Heavy mix of `NavigationView`` (109) and `NavigationStack`` (29), plus custom deep-link routing across tab-level stacks.

****Why it is wrong****

- Inconsistent push behavior, back-stack differences, and harder state restoration.

****Recommended fix****

- Standardize on `NavigationStack`` + typed routes and central route mapping.

H5. Asynchronous trigger density and race risk

****What is wrong****

- ~326 asynchronous trigger points (`.task``, `.onAppear``, delayed dispatches, detached tasks).

****Why it is wrong****

- Increased chance of double-fetches, stale state updates, and difficult-to-reproduce lifecycle bugs.

****Recommended fix****

- Introduce flow ownership rules: one source of truth per screen/domain for network refresh and task cancellation strategy.

H6. Automated test coverage is too low for system complexity

****What is wrong****

- Only two test-like Swift files discovered (`LimitationFilterTests.swift``, `TestSmartSelection.swift``).

****Why it is wrong****

- High risk of regressions in auth/onboarding/challenges/deep links where behavior is stateful and async.

****Recommended fix****

- Add a minimum regression suite: auth onboarding transitions, workout resume, challenge invite/accept flows, shared link open, and critical SQL RPC contract tests.

Medium

M1. UI design system is partially centralized but still fragmented

****What is wrong****

- Shared styling exists (`AdaptiveColors``, `SleekCardBackground``), but parallel ad-hoc style systems also exist (e.g., custom card modifiers in `ContentView``), with repeated gradient/styling code across views.

****Why it is wrong****

- Leads to subtle inconsistencies across tabs and harder visual refactors.

****Recommended fix****

- Define a strict design token + component library (cards/buttons/headers/backgrounds) and ban local one-off styles except with explicit exceptions.

M2. Documentation drift

****What is wrong****

- `README.md` describes older project structure/naming (BuiltSimple) and does not reflect current module layout and production architecture.

****Why it is wrong****

- Slows onboarding and increases misunderstanding of current system boundaries.

****Recommended fix****

- Update README to current architecture map, ownership boundaries, and deployment process.

M3. Edge function error semantics are inconsistent for clients

****What is wrong****

- `usda-food-search` returns HTTP 200 for catch-all errors with error payload.

****Why it is wrong****

- Makes client-side reliability and observability harder; true failures can be silently interpreted as empty success.

****Recommended fix****

- Return typed error codes with proper HTTP statuses, and map expected “empty results” separately from actual server failure.

M4. Known TODOs in functional paths

****What is wrong****

- TODOs remain in live paths (`WorkoutSharingService`, `SharedWorkoutView`, SQL comments for friend request notification queueing).

****Why it is wrong****

- Indicates partially delivered UX where users may expect behavior that is not complete.

****Recommended fix****

- Track TODOs in a release checklist; block feature labeling until critical TODOs are closed.

4) User Flow Review (cross-feature)

Onboarding/Auth

- Overall flow is comprehensive and robust; however complexity is very high and spread across large files and notifications.
- Main risk: race/duplication from multiple startup/auth triggers.

Workout creation and active session

- Strong persistence safeguards in `WorkoutManager` (state restore, corruption protection).
- Risk remains around complexity and lifecycle-triggered side effects.

Social/challenges/realtime

- Realtime architecture is strong and event-driven.
- Risk lies in backend function overlap and migration drift affecting challenge/friend correctness over time.

Nutrition

- Rich feature set and cache-backed architecture.
- Key reliability risk is error handling semantics and API key management.

Integrations (HealthKit/Strava/Fitbit/InBody)

- Broad and thoughtful support.
- Needs stronger separation of secrets and cleaner coordination of sync triggers.

5) Recommended Remediation Plan

Phase 1 (Immediate: 1-3 days)

1. Rotate all exposed secrets and remove secrets from repo/client.
2. Stabilize shared workout deep-link cloud fetch path.
3. Freeze new SQL hotfix scripts; establish one canonical migration baseline.

Phase 2 (Short term: 1-2 weeks)

1. Consolidate duplicated SQL function definitions into migration chain.
2. Standardize navigation strategy (`NavigationStack` typed routes).
3. Replace raw prints with structured logger + redaction.
4. Update README and architecture docs to current reality.

Phase 3 (Mid term: 2-4 weeks)

1. Break down top 5 oversized files by domain boundaries.

2. Add critical-path automated tests (auth/workout/challenge/deeplink).
3. Add CI checks for migration ordering/schema drift and secret scanning.

6) Strengths Observed

- Excellent product ambition and breadth.
- Good use of staged startup/performance awareness.
- Thoughtful persistence safeguards in workout flow.
- Realtime social update architecture is directionally strong.

7) Audit Metrics Snapshot

- Swift files scanned: **238**
- SQL files scanned (`supabase/`): **31**
- Largest Swift file size: **8,727 lines**
- Async trigger points (`.task`, `.onAppear`, detached tasks, delayed dispatch): **326**
- `print(...)` call sites: **3,485**
- `NavigationView` occurrences: **109**
- `NavigationStack` occurrences: **29**
- SQL function names defined: **51**
- Duplicated function definitions across SQL files: **14**

Final Assessment

The app is feature-rich and close to a high-quality platform product, but current risk concentration is in **security hygiene**, **architecture scale management**, and **backend migration discipline**. Addressing the critical and high-priority findings above will significantly reduce production risk and improve delivery velocity.